Magpie — Sprint 9 Technical Spec (Robustness)

Audience: implementing agent (Codex) + Regan. **Builds on** Sprints 0–8. **Status: planning / review — implement after approval.**

> This sprint changes **how Magpie runs**, not what it does. It replaces the two hand-managed dev servers with one stable Windows service: a production WSGI server (Waitress) serving the API, the admin, and the built React SPA from a **single LAN port**, run under NSSM so it starts on boot and restarts on crash. No database migration, no Linux, no Docker, no HTTPS, no app features. `restart servers.txt` **is deleted at the end** — its job is now the service's.

Confirmed environment (F-14 + Regan 2026-06-10): Windows 10 / PowerShell 7.4; **SQLite** (`db.sqlite3`) stays the runtime DB; media at `…\backend\media_files` ; single-operator, LAN-only. **Reached at** `http://localhost:8000` **and** `http://192.168.1.86:8000` (IP confirmed: `192.168.1.86`).

---

0. Locked decisions this spec implements

- **D-82** End-state **A** affirmed by Regan: Magpie remains a **Windows + SQLite single-operator LAN tool**. **No** Linux migration, **no** Postgres migration, **no** Docker. One Sprint 9 (no split).
- **D-83** Production app server = **Waitress** (pure-Python, Windows-native), replacing `manage.py runserver` .
- **D-84** Frontend **built** (`npm run build`) and served **statically via WhiteNoise** from the same Django process; **one LAN port** serves SPA + `/api` + `/admin` . The Vite dev server and dev proxy are retired as the runtime. The production build calls the API **same-origin** (`/api`) — no proxy, no hardcoded `localhost:8000` .
- **D-85** Process management = **NSSM** Windows service ("Magpie"): **start-on-boot + auto-restart on crash**. Replaces the two-dev-server dance and `restart servers.txt` .
- **D-86** Production-settings hygiene **now**: `DEBUG=False` , `ALLOWED_HOSTS` , `CSRF_TRUSTED_ORIGINS` , `SECRET_KEY` from env, `collectstatic` / `STATIC_ROOT` + WhiteNoise. **Deferred to HTTPS:** `SESSION/CSRF_COOKIE_SECURE` , HSTS, SSL-redirect

(set off now, flip with HTTPS).

- **D-87** CI parity: add a **SQLite lane** alongside the existing Postgres lane — the suite runs on **both** (SQLite = ships, Postgres = stricter checks). Both green to pass.
- **D-88** Closure gate (operational): reboot-survival **and** crash-restart proof, plus the full checklist in §8; `DEPLOYMENT.md` + `ROBUSTNESS_EVIDENCE.md`; the Sprint 8 backup must still run against the new layout.
- **D-89** HTTPS / public exposure remain a **documented runbook only**; out-of-scope list in §10.

---

1. Target architecture

Before: `runserver` on `:8000` (API/admin) + Vite dev server on `:5173` (SPA) + a dev proxy; two processes, started by hand. **After:** one **Waitress** process bound to `0.0.0.0:8000`, running the Django WSGI app, which serves:

- `/api/*` → DRF
- `/admin/*` → Django admin
- hashed static assets + Django/admin static → **WhiteNoise**
- everything else → the SPA `index.html` (client-side routing)

…run as the **NSSM service "Magpie"** (auto-start, auto-restart). Reached at `http://localhost:8000` (host) and `http://192.168.1.86:8000` (LAN). Binding `0.0.0.0` is intended (LAN devices reach it); safe because the router forwards nothing inbound (LAN-only).

---

2. App server — Waitress

- New dependency: **`waitress`** (`pip install waitress`; pin in `requirements.txt`; pure-Python, no binary).
- A launcher `backend/serve.py` (or a documented `waitress-serve` invocation) that loads the Django WSGI application and serves it:

```
# serve.py (sketch)
import os
os.environ.setdefault("DJANGO_SETTINGS_MODULE", "config.settings")  # producti
```

```
from waitress import serve
from django.core.wsgi import get_wsgi_application
app = get_wsgi_application()
serve(app, host="0.0.0.0", port=int(os.environ.get("MAGPIE_PORT", "8000")), th
```

- Threads: a small pool (4) is ample for single-user. Port via `MAGPIE_PORT` (default 8000).
- **Env loading:** the launcher must load the same env the manual run uses (`DATABASE_URL`, `MAGPIE_TOKEN_ENCRYPTION_KEY`, `EBAY_*`, `SECRET_KEY`, `DEBUG`, `ALLOWED_HOSTS`, `MAGPIE_BACKUP_PASSPHRASE`, etc.) — via the app's existing `.env` mechanism, **not** the interactive shell (a Windows service does not inherit your PowerShell session env). Confirm the service process sees identical env to the manual run.

---

3. Frontend build + WhiteNoise (one origin)

1. **Build:** `npm run build` → `frontend/dist/`. Ensure Vite `base: '/'` and the **API client uses a relative `/api` base** in the production build (this is the silent-breakage point — a hardcoded `http://localhost:8000` works on the host but fails from `192.168.1.86`).
2. **WhiteNoise:** add `whitenoise.middleware.WhiteNoiseMiddleware` (just below `SecurityMiddleware`); set `STATIC_ROOT`, `STATICFILES_STORAGE = "whitenoise.storage.CompressedManifestStaticFilesStorage"`. New dependency: **`whitenoise`** (pure-Python; pin it).
3. **Serving the SPA:** copy/collect `dist/` assets so WhiteNoise serves the hashed JS/CSS, and add a **catch-all** urlconf entry (LAST, after `/api` and `/admin` and static) returning the built `index.html` so client-side routes resolve. Verify deep-linking (e.g. reloading `/inventory/<id>`) returns `index.html`, not a 404.
4. `collectstatic` runs as part of the build/deploy step (gathers Django + admin + the SPA assets into `STATIC_ROOT`).

---

4. NSSM Windows service

- Install NSSM (portable binary on PATH — the one new operational tool, like `rclone`).
- Register (documented exact commands in `DEPLOYMENT.md`):

```
nssm install Magpie "<venv>\Scripts\python.exe" "<repo>\backend\serve.py"
nssm set Magpie AppDirectory "<repo>\backend"
nssm set Magpie Start SERVICE_AUTO_START          # start on boot
nssm set Magpie AppStdout "<repo>\backend\logs\service-stdout.log"
nssm set Magpie AppStderr "<repo>\backend\logs\service-stderr.log"
nssm set Magpie AppExit Default Restart           # auto-restart on crash
nssm set Magpie AppThrottle 5000                  # ms; avoid tight crash loop
```

(Env via the `.env` mechanism per §2, or `nssm set Magpie AppEnvironmentExtra` for the critical names — pick one and document it.)

- **Lifecycle documented:** `nssm start|stop|restart Magpie`, `nssm remove Magpie confirm`, **and the update procedure (stop → git pull → pip/npm install →** `npm run build` **→** `collectstatic` **→ migrate → start).**
- **Delete** `restart servers.txt` **— superseded.**

---

5. Production settings hygiene (D-86)

Env-driven (production run sets these; dev can differ):

- `DEBUG=False`.
- `ALLOWED_HOSTS = localhost,127.0.0.1,192.168.1.86` **(env-driven; extend if the hostname is added later).**
- `CSRF_TRUSTED_ORIGINS = http://localhost:8000,http://127.0.0.1:8000,http://192.168.1.86:8000` **(scheme+host+port; same-origin SPA so this covers the app).**
- `SECRET_KEY` **from env (confirm it already is; if a dev default exists, generate a real one and store it out-of-band with the other secrets — Sprint 8 D-78 list).**
- `STATIC_ROOT` **set; WhiteNoise compressed-manifest storage.**
- **Deferred (set OFF now, comment that they flip with HTTPS):** `SESSION_COOKIE_SECURE=False`, `CSRF_COOKIE_SECURE=False`, `SECURE_HSTS_SECONDS=0`, `SECURE_SSL_REDIRECT=False`**. Forcing these on without HTTPS would break the LAN HTTP app.**
- **Sanity:** `python manage.py check --deploy` **run and its output noted (expected to flag the deferred HTTPS items — that's acceptable and documented, not a blocker).**

---

6. CI — add the SQLite lane (D-87)

- In the validation workflow, run the suite on **both** databases: keep the Postgres job; add a **SQLite job** (`DATABASE_URL=sqlite:///...` on a temp file). Both must be green.
- Rename the workflow display name to **"Validation"** (it's no longer Postgres-only) with two jobs (`sqlite` , `postgres`) — or keep the file and add the lane; document whichever. This proves the **shipped** DB (SQLite), not just the stricter one.
- Add a lightweight **production-mode smoke** (CI step or test): with `DEBUG=False` + `collectstatic` , start Waitress, `GET /` returns the SPA `index.html` (200), `GET /api/<a simple endpoint>` returns 200, a deep-link path returns `index.html` . Catches the same-origin/static/catch-all wiring.

---

7. `DEPLOYMENT.md` (runbook)

Install/build steps; the NSSM commands (§4); start/stop/restart/update procedure; the URL (`http://192.168.1.86:8000` , `http://localhost:8000`) + port note (80 optional for a cleaner URL, with the elevation caveat); env-var **names** list (cross-ref Sprint 8 manifest); where logs + backups live; and an **"If you ever expose Magpie to the internet" appendix** (front the port with **Caddy** auto-HTTPS or a **Cloudflare Tunnel**, then flip `*_COOKIE_SECURE` /HSTS/SSL-redirect on, restrict `/admin` , lock `ALLOWED_HOSTS`). Built, deferred — not executed.

---

8. Acceptance criteria (D-88 — operational, headline = reboot + crash-restart)

In `ROBUSTNESS_EVIDENCE.md` :

1. App runs through **Waitress**, not `runserver` .
2. Frontend is the **built static bundle** (WhiteNoise), not the Vite dev server.
3. **One LAN port** serves SPA + `/api` + `/admin` ; verified at `http://192.168.1.86:8000` from another LAN device + `http://localhost:8000` on the host; SPA deep-link reload works.
4. `DEBUG=False` verified (e.g. a deliberately-bad URL shows the plain 404/500, not the debug page).

5. `ALLOWED_HOSTS` + `CSRF_TRUSTED_ORIGINS` verified (app works from `192.168.1.86`; an unlisted Host is rejected).
6. **NSSM service starts on boot** (reboot the machine; the app is up with no manual step).
7. **Crash-restart proven** — kill the Waitress/python process; NSSM brings it back within the throttle window; the app recovers.
8. The **Sprint 8 backup still runs** against the new layout (run `manage.py backup`; archive produced; a spot restore or at least the produced-archive + scheduled-task-still-fires confirmed).
9. **Dual SQLite + Postgres CI green.**
10. `DEPLOYMENT.md` + `ROBUSTNESS_EVIDENCE.md` exist; `restart servers.txt` removed.

---

9. Test plan

- **CI (automated):** dual-DB lanes green; the production-mode smoke (DEBUG=False + collectstatic + Waitress serves `/`, `/api`, deep-link). `manage.py check --deploy` runs (deferred-HTTPS warnings expected/documented).
- **Operational (manual, evidence file):** reboot-survival (#6), crash-restart (#7), LAN access from a second device (#3), DEBUG=False behaviour (#4), Host rejection (#5), backup-against-new-layout (#8).
- No new migrations expected; no model/feature changes.

---

10. Out of scope (D-89)

Linux migration · Postgres runtime migration · Docker · HTTPS execution · public/internet exposure (runbook only) · secure-cookie/HSTS/SSL-redirect activation · multi-user/RBAC · monitoring/APM · CDN · any application feature (Phase 5/6 untouched) · changing the backup mechanism beyond pointing it at the new layout.

---

11. Seed

No seed changes. No data touched (SQLite stays; no migration).

Hand-off notes

- **The same-origin `/api` base in the production build is the #1 silent-breakage risk** — verify the built SPA calls `/api` relatively, not `localhost:8000`, or it works on the host and fails from the phone.
- A Windows **service does not inherit your shell env** — make sure the service loads the real env (`.env` or NSSM `AppEnvironmentExtra`); a missing `MAGPIE_TOKEN_ENCRYPTION_KEY` or `DATABASE_URL` under the service is a classic NSSM gotcha.
- Crash-restart is part of the gate, not optional — prove NSSM revives a killed process.
- Do **not** migrate the DB or change how the app behaves — only how it's served. If you're editing models or features, you've left the sprint.
- HTTPS stays a runbook. Don't flip the secure-cookie/HSTS/SSL settings on — they'd break the LAN HTTP app.
- When ambiguous, prefer the smaller change that satisfies §8.